

Advanced AI with Security

Comprehensive journey through modern AI techniques and security implications.



About the Instructor

Daniel Blanco



[Daniel Blanco - | LinkedIn](#)

- Daniel Blanco
AI & Platform Engineering Instructor
- 15+ years in IT
MBA | AWS Solutions Architect | CKA
5+ years technical training experience
- I focus on building secure, production-ready AI and cloud systems — and teaching engineers how to do the same.
- Southern California

What Is Generative AI?



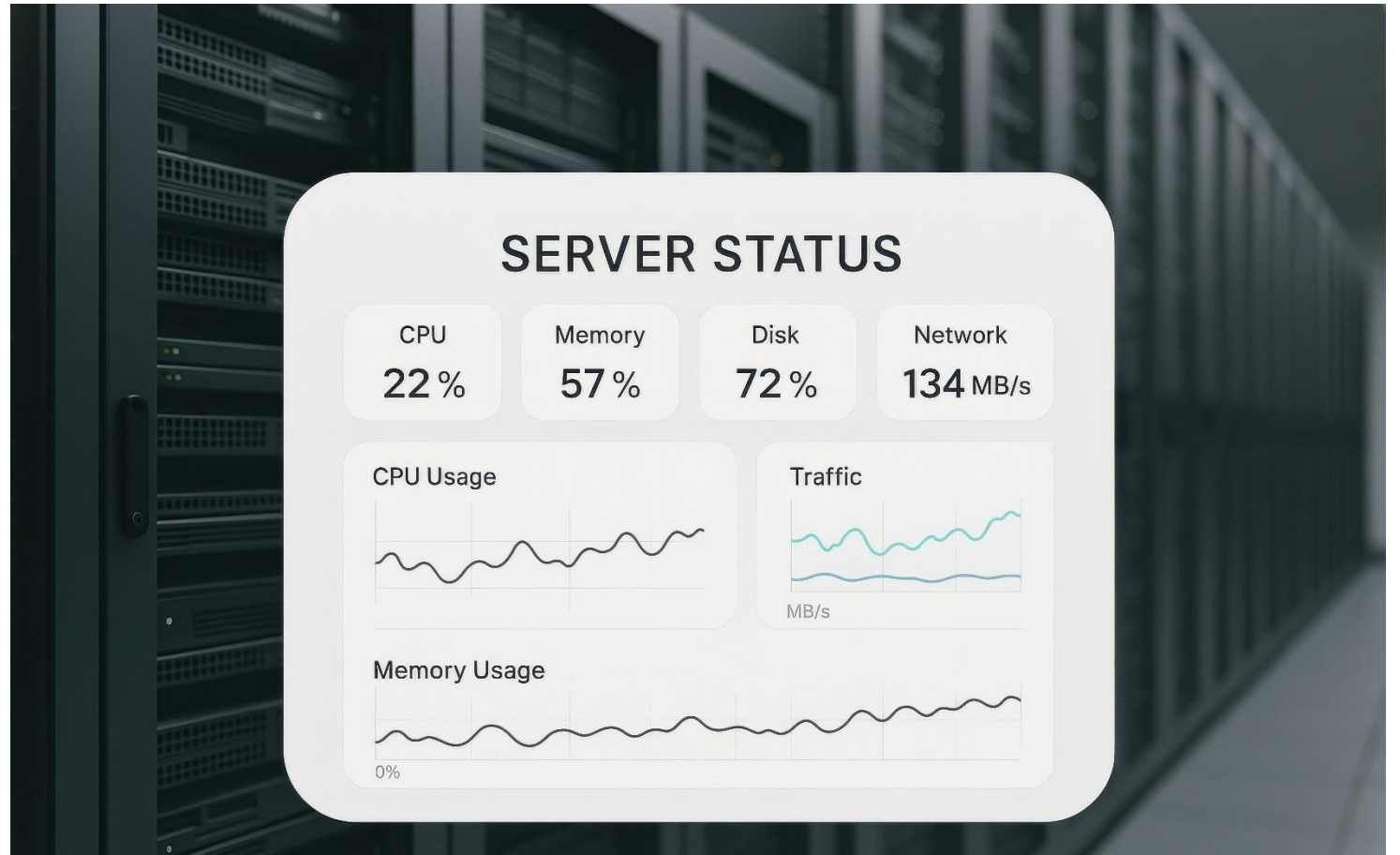
Generative AI creates new content based on patterns learned from training data. Unlike traditional ML which classifies or predicts, generative models produce text, images, code, and more.

- Learns patterns from massive datasets
- Generates new content token by token
- Foundation: transformer neural networks

Foundation Models

Large pre-trained models adapted for many tasks. Pre-trained on broad internet data then fine-tuned or prompted.

- Billions of parameters, weeks of training
- Transfer learning across diverse tasks
- Base for enterprise AI applications



Tokens and Tokenization

```
SYSTEM LOG V2.5.1 - 04/23/1986
-----
[08:15:21] USER 'ADMIN' LOGGED IN SUCCESSFULLY FROM TERMINAL #3.
[08:15:25] INITIALIZING MODULE 'CORE_SERVICES.MOD' -- STATUS: OK
[08:15:30] INITIALIZING MODULE 'NETWORK_CLIENT.MOD' -- STATUS: PING_P
[08:15:35] INITIALIZING MODULE 'NETWORK_CLIENT.MOD' -- STATUS: OK. CO
[08:15:40] LOADING CONFIGURATION FILE 'C:\SYSTEM\CONFIG.INI' FROM DIS
[08:15:45] LOADING MODULE 'DATABASE_MANAGER.MOD' -- STATUS: OK. CONNE
[08:15:50] QUERY: SELECT * FROM 'INVENTORY' WHERE 'ID' = '245'. 1 ROW
[08:16:00] DISPLAYING RESULTS TO TERMINAL #3.
[08:16:15] USER 'ADMIN' ISSUED COMMAND: 'VIEW LOGS'. ACCESS GRANTED.
[08:16:30] USER 'ADMIN' LOGGED OUT. SESSION TERMINATED.
[08:16:35] SYSTEM IDLE. WAITING FOR NEW USER LOGIN.
-----

+-----+
| INVENTORY REPORT APRIL 1986
+-----+
| ID | ITEM DESCRIPTION          | QUANTITY | LOCATION | STATUS
+-----+-----+-----+-----+
| 01 | PENTIUM PROCESSOR 8086    | 000185   | A1.04    | IN STOCK
| 02 | MFM HARD DISK DRIVES      | 000003   | A1.01    | LOW STOCK
| 03 | 5.25" FLOPPY DISKETTES    | 001250   | B3.07    | AVAILABLE
| 04 | CRT MONITORS (GREEN)      | 000012   | C2.11    | IN STOCK
| 05 | KEYBOARD SETS W/ CABLE    | 000020   | C2.15    | IN STOCK
| 06 | POWER SUPPLY UNITS        | 000007   | A1.04    | LOW STOCK
| 07 | RS-232 CABLES             | 000500   | B3.08    | IN STOCK
+-----+-----+-----+-----+

                                     PAGE 1 of 1

Volume in drive C has no label
Volume Serial Number is A8E0-607B

Directory of C:\

12/11/1985  08:30 AM                <DIR>          DATA
12/11/1985  08:31 AM                <DIR>          DOCS
12/11/1985  08:32 AM                <DIR>          GAMES
12/11/1985  08:35 AM                <DIR>          SYSTEM
```

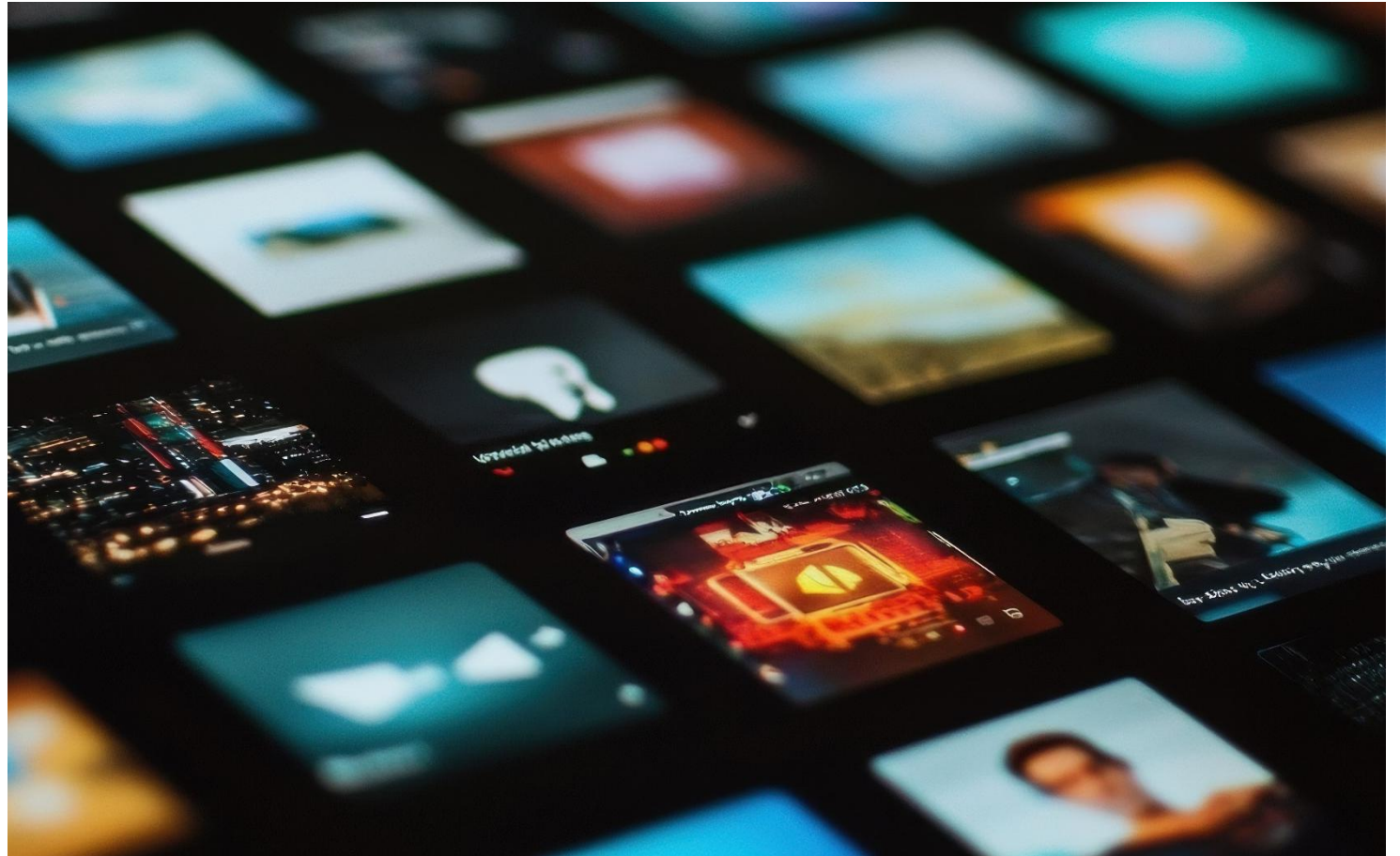
Text is converted to tokens before processing. A token is roughly 0.75 words, or about 4 characters. The tokenization process is deterministic and model-specific.

- Text splits into subword tokens
- GPT typically: ~1.3 tokens per word
- Affects input/output costs and limits

Context Windows

Maximum number of tokens the model can accept as input plus generate as output. Larger windows enable processing longer documents.

- GPT-5.3-Codex: 128K tokens context
- Claude Opus 4.6: 1M tokens
- Limits affect cost and capability



Temperature and Sampling



Temperature (0.0-2.0) controls output randomness. Low temperature = deterministic; high = creative. Sampling methods like Top-P limit the token probability pool.

- 0.0: Always pick most likely token
- 1.0: Balanced between predictability and variety
- 2.0: Maximally random from broader pool

LLMs and Multi-Modal Models

LLMs process text only. Multi-modal models process text, images, audio, or video. Multi-modal capabilities enable richer enterprise applications.

- Text-only LLMs: fast, cost-effective
- Multi-modal: vision reasoning added
- Future: integrated sensory reasoning



AI Provider Landscape



Major providers: OpenAI (GPT-5.3-Codex Feb 2026), Anthropic (Claude Opus 4.6, 1M context), Google (Gemini 3 Pro), Meta (Llama 4 family), AWS (Amazon Nova 2). Each optimizes differently.

- OpenAI: GPT-5.3-Codex latest, available in Azure
- Anthropic Claude: Long context, safety focus
- Google/Meta/AWS: Open and proprietary options

Provider Data Security

Different providers have different data handling policies. Understand where uploaded data goes, training practices, and compliance certifications.

- OpenAI: No training on API data
- Anthropic: No training on API data
- Enterprise agreements vary by vendor



Enterprise Data Boundaries



Critical question: where does your uploaded data go? Risk of using in model training, shared with third parties, or stored indefinitely. Sensitive data requires SOC 2 or FedRAMP providers.

- Consumer tools often train on data
- Enterprise tiers offer data isolation
- Compliance depends on data classification

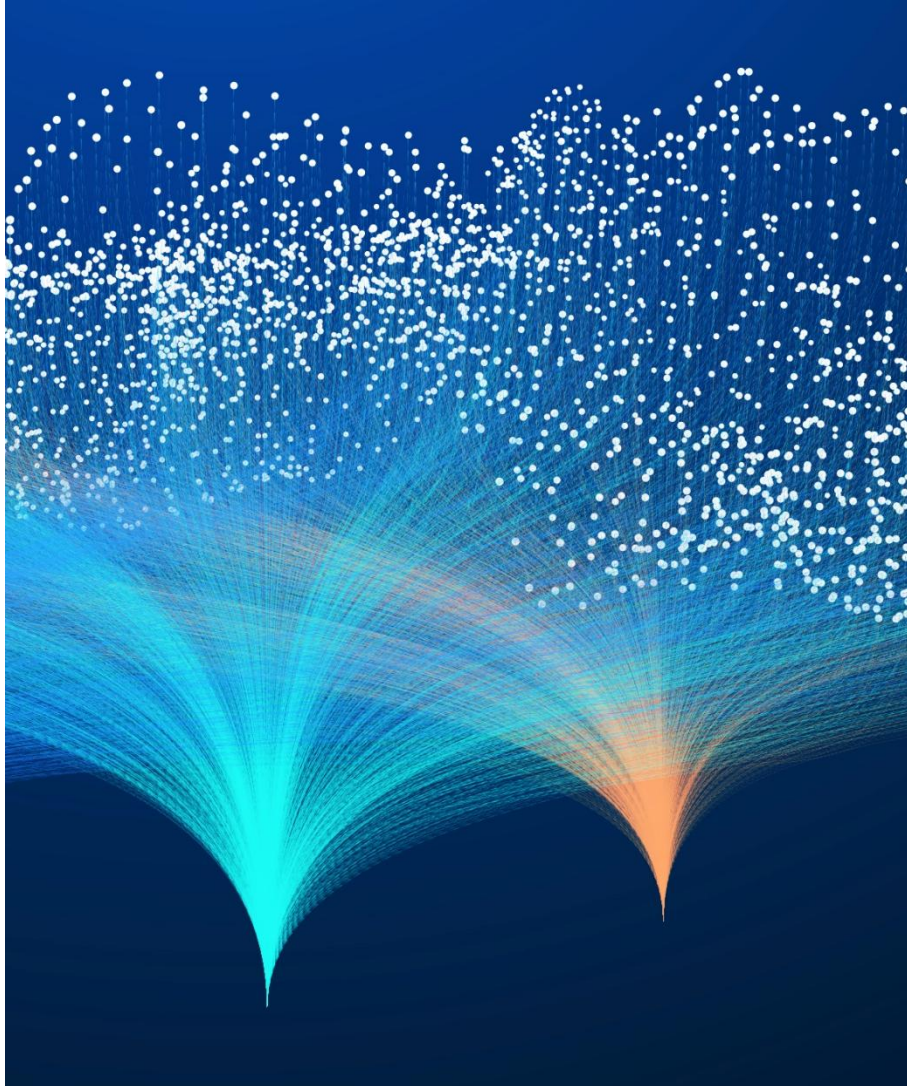
Prompt Engineering Fundamentals

Well-crafted prompts narrow the possibility space, making outputs more relevant. Vague prompts lead to inconsistent, off-topic responses.

- Specific instructions reduce ambiguity
- Context and examples guide behavior
- Iterative refinement improves quality



Prompt Engineering: Examples



Bad prompt: 'Tell me about AI.' Good prompt: 'Explain how transformer models work in 3 sentences for a high school audience.' Specificity, audience, and constraints matter.

- Bad: vague, open-ended, ambiguous targets
- Good: clear goal, constraints, audience level
- Examples: role assignment, few-shot patterns

Structured Prompting Patterns

Few-shot learning: include examples.
Chain-of-Thought: ask model to explain reasoning. Role-based: assign persona or expertise.

- Few-shot: Show examples, then new query
- Chain-of-Thought: 'Think step-by-step'
- Role-based: 'You are an expert....'



RAG and Agents Preview



RAG (Retrieval-Augmented Generation): Model retrieves relevant documents before generating. Agents: Models call tools and take actions. Both extend LLM capabilities beyond raw knowledge.

- RAG: Solve knowledge cutoff problem
- Agents: Enable real-time tool integration
- Labs 2-3 dive deep into both

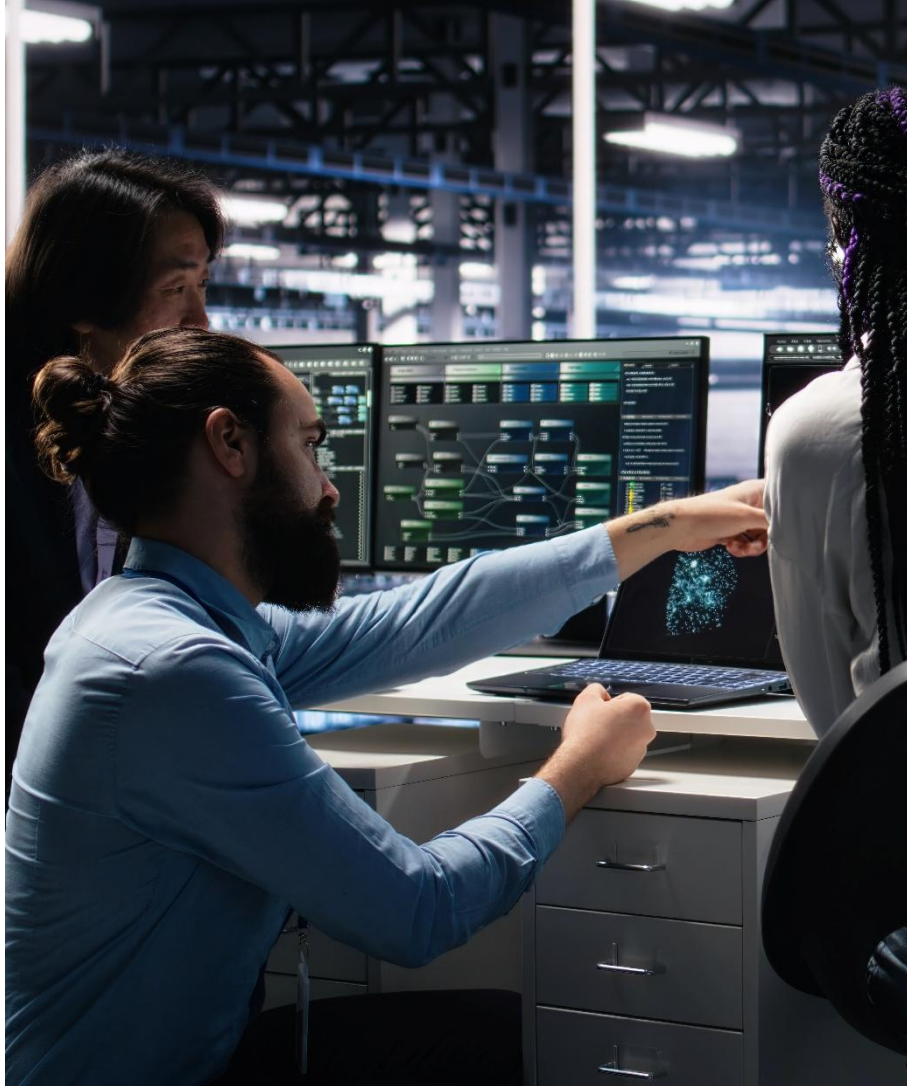
Lab 1.1: Compare AI Tools

Try the same prompts across OpenAI, Anthropic, Google, and GitHub Copilot. Observe differences in quality, speed, and safety.

- Test identical prompts on each platform
- Document response quality and latency
- Note differences in model behavior



Lab 1.1: Data Security Exercise



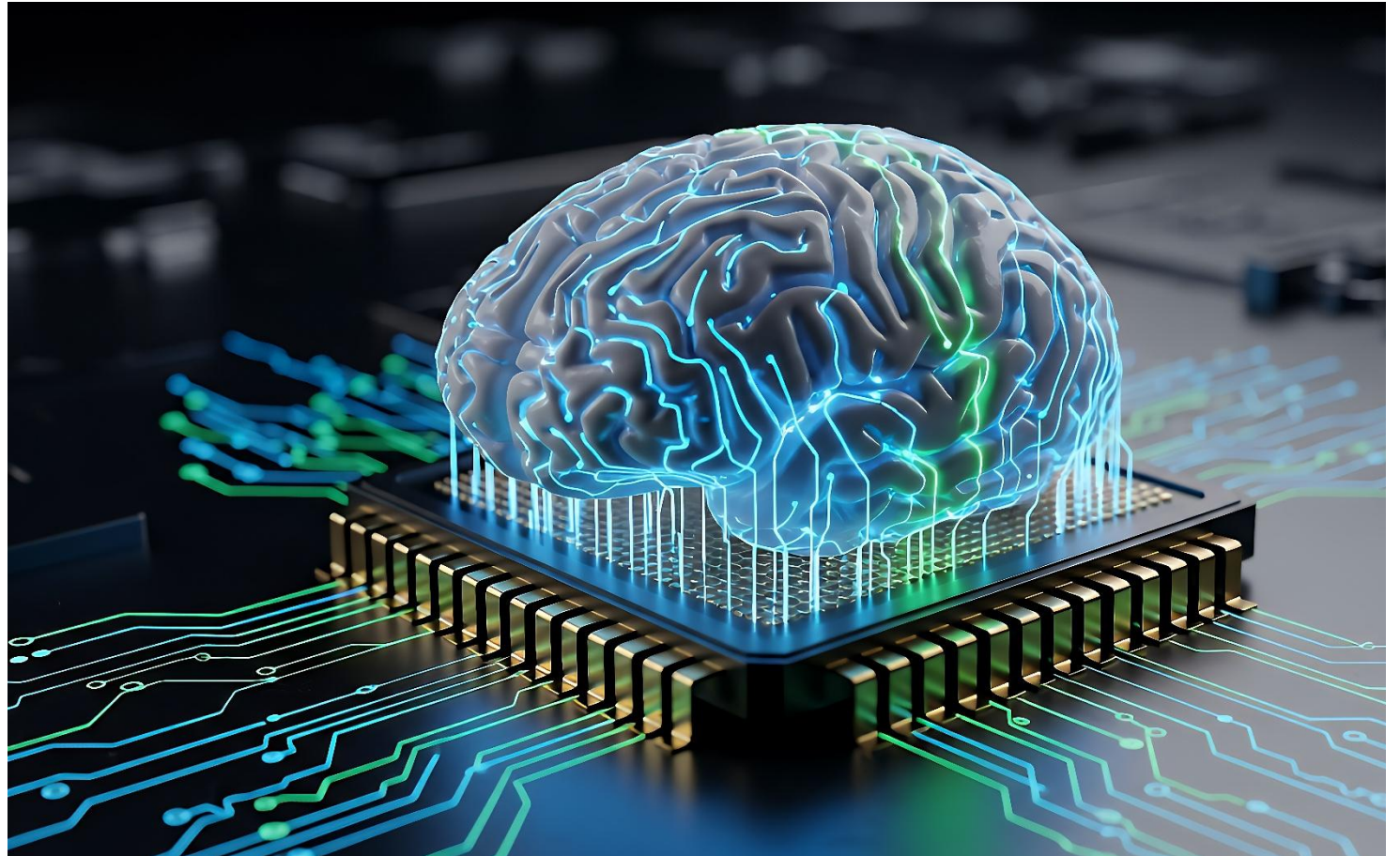
Upload a CSV with sensitive sample data to a consumer AI tool and enterprise tool. Document where data goes, whether it could be trained on, and compliance implications.

- Upload to OpenAI ChatGPT and Azure
- Check terms of service for each
- Compare data handling policies

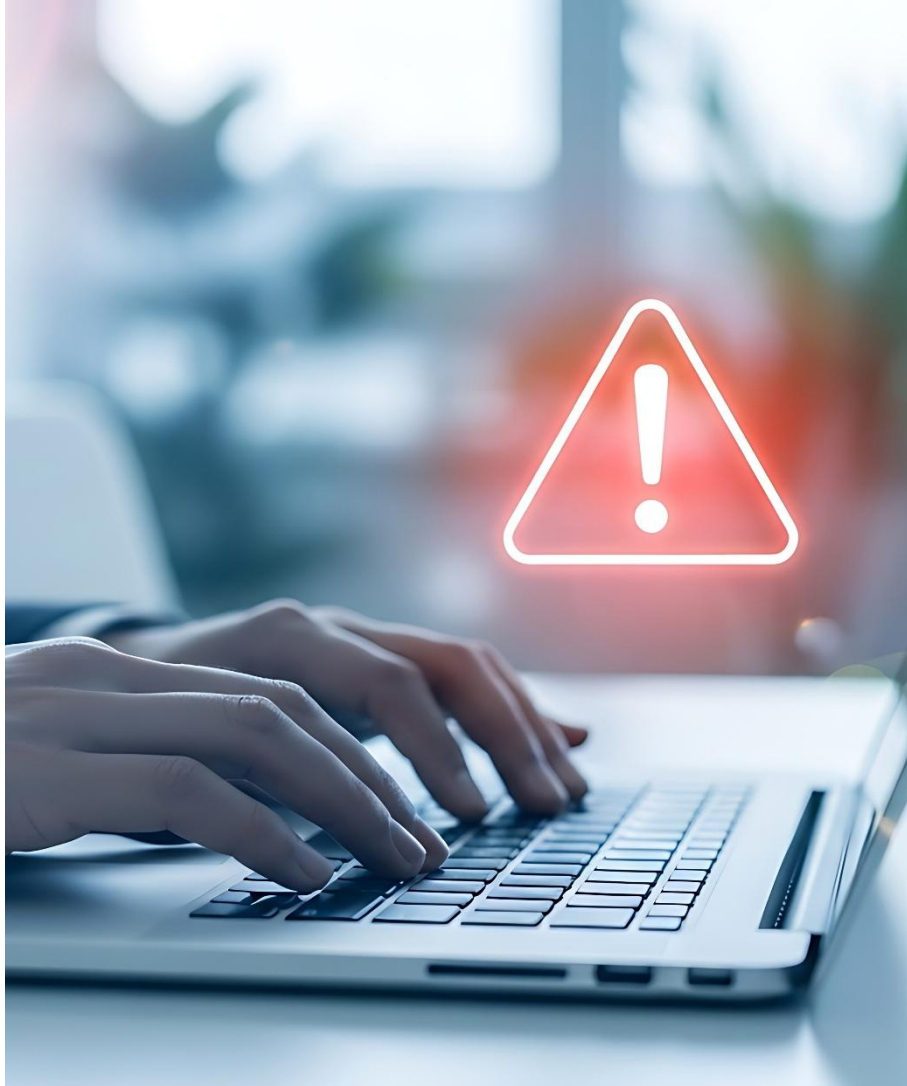
Lab 1.1: Prompt Refinement

Start with a vague prompt. Iteratively add constraints, examples, and specificity. Observe quality improvements at each step.

- Vague → Structured → Few-shot
- Add audience and format constraints
- Measure improvement in output quality



Lab 1.1 Summary



Key takeaways: Foundation models power generative AI. Tokens determine cost and limits. Temperature controls randomness. Provider data security matters. Prompt engineering is a core skill.

- Understand token economics and context limits
- Know your provider's data practices
- Prompt engineering drives output quality

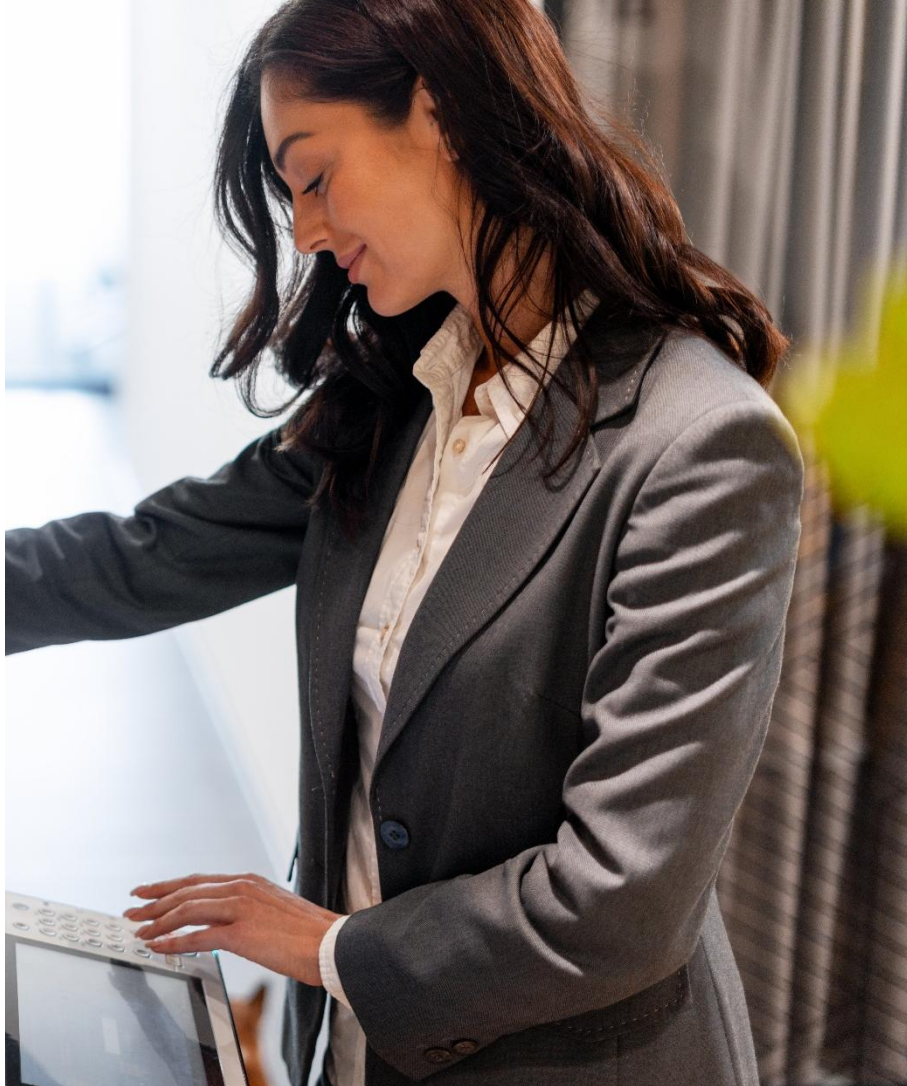
Why Azure AI Foundry?

Enterprise AI platform with governance, security, and compliance. Unlike ChatGPT, Foundry provides RBAC, audit logs, and content filters.

- Governance: RBAC, audit trails, policies
- Security: encryption, data boundaries
- Compliance: SOC 2, FedRAMP options



Azure Resource Hierarchy



Subscriptions contain resource groups, which contain resources like storage, compute, and AI services. This hierarchy enables billing and access control.

- Subscription: billing and cost boundary
- Resource Group: logical grouping of resources
- Resources: storage, AI Hub, compute

Storage Accounts and Blobs

Cloud file storage for AI datasets and documents. Blobs are objects in containers with REST API access and fine-grained access control.

- Containers organize related blobs
- Access via SAS tokens or managed identity
- Scalable to exabytes of data



Data Classification in Azure



Public, internal, confidential, restricted tiers. Azure enforces encryption at rest and in transit. Sensitivity labels control who accesses data. Keys stay in customer control with CMK.

- Encryption: at rest, in transit, in use
- Sensitivity labels guide access
- Customer-managed keys (CMK) option

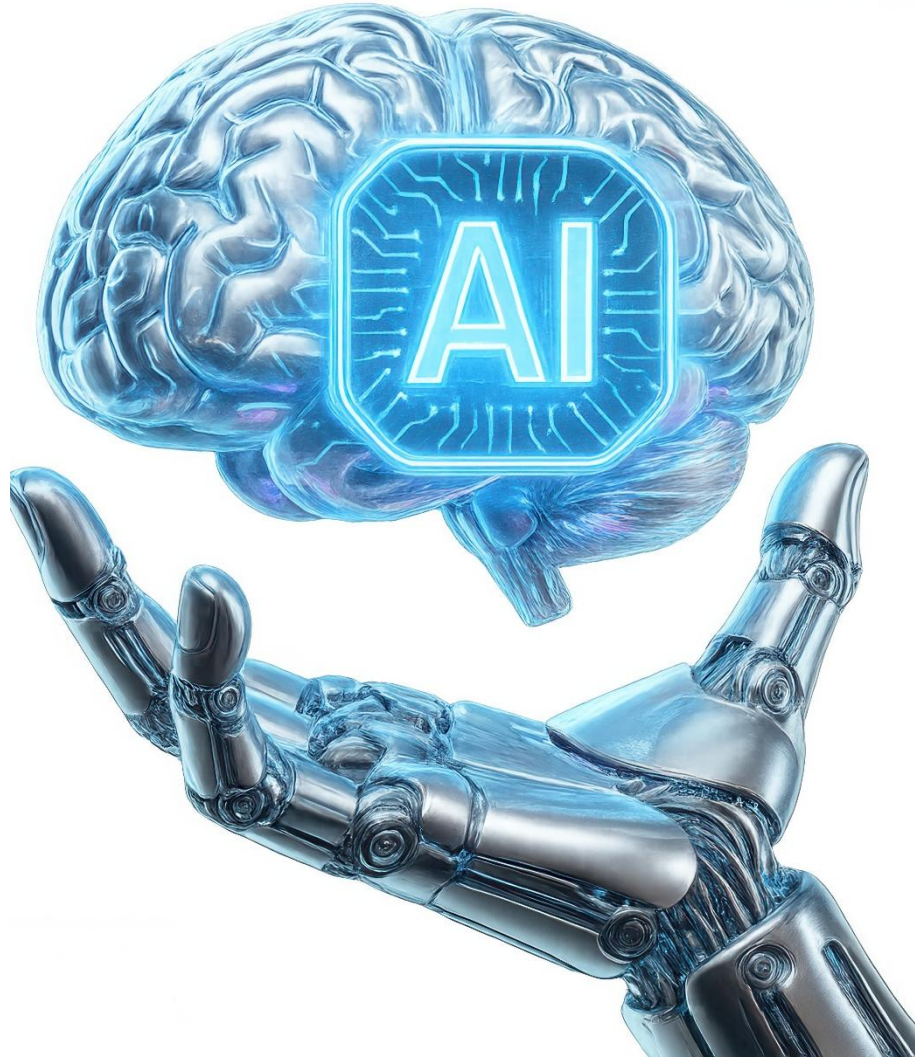
Role-Based Access Control

RBAC assigns permissions to users and service principals. Built-in plus custom roles. Least privilege limits exposure.

- Owner: full control
- Contributor: create and modify
- Reader: view-only access



Azure AI Foundry Platform



Hub: organization-level settings. Projects: isolated workspaces for teams. Model catalog: access to GPT-5.3-Codex, Llama 4, and open models. Playground: test prompts before deployment.

- Hub: shared settings and governance
- Projects: team isolation and billing
- Catalog: 100+ models available

Model Catalog and Playground

Access models from OpenAI, Meta, Mistral, and Cohere. Playground tests prompts live. Compare outputs side-by-side.

- GPT-5.3-Codex, Llama 4, Mistral available
- Test prompts in playground instantly
- Deploy to scalable endpoints



System Messages



Define agent behavior, persona, constraints, and rules. System message is prepended to every conversation. Controls guardrails without code changes.

- Set agent role and expertise level
- Define safety constraints and guardrails
- Influence response tone and style

Foundry vs Consumer AI

Consumer AI: no audit logs, data may train models. Enterprise Foundry: audit trails, no training on data, SOC 2, HIPAA, RBAC.

- Audit logs and compliance reporting
- Data isolation and no training
- Governance and policy enforcement



Content Filters



Hate, violence, sexual, self-harm each have severity levels (low, medium, high). Foundry blocks responses exceeding threshold. Configurable per use case.

- Hate speech: detect and block
- Violence: prevent harmful instructions
- Sexual: filter inappropriate content

Jailbreak Detection

Detects prompt injection attempts to bypass guardrails. Identifies patterns trying to override system messages or trigger unintended behavior.

- Identifies override attempts and tricks
- Prevents 'ignore previous instructions'
- Logged for security analysis



Custom Blocklists



Define organization-specific terms or patterns to always filter. Combine with built-in filters. Useful for preventing sensitive data exposure or competitive information.

- Add domain-specific restricted terms
- Combine with hate/violence/sexual filters
- Logged for audit compliance

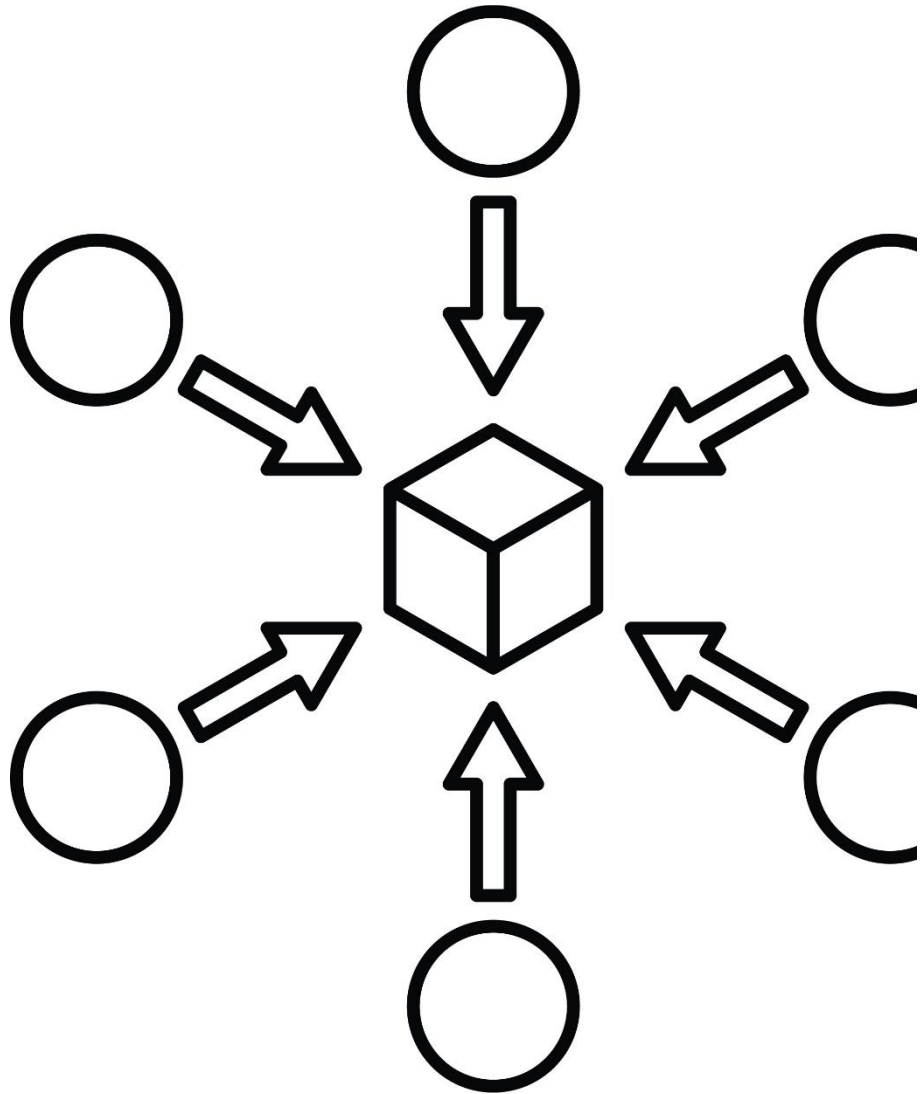
Azure Policy for AI

Enforce restrictions at organization level: which models allowed, which regions required, which SKUs permitted. Prevents unauthorized deployments.

- Restrict models by vendor or type
- Enforce data residency requirements
- Control deployment SKU options



VS Code Integration



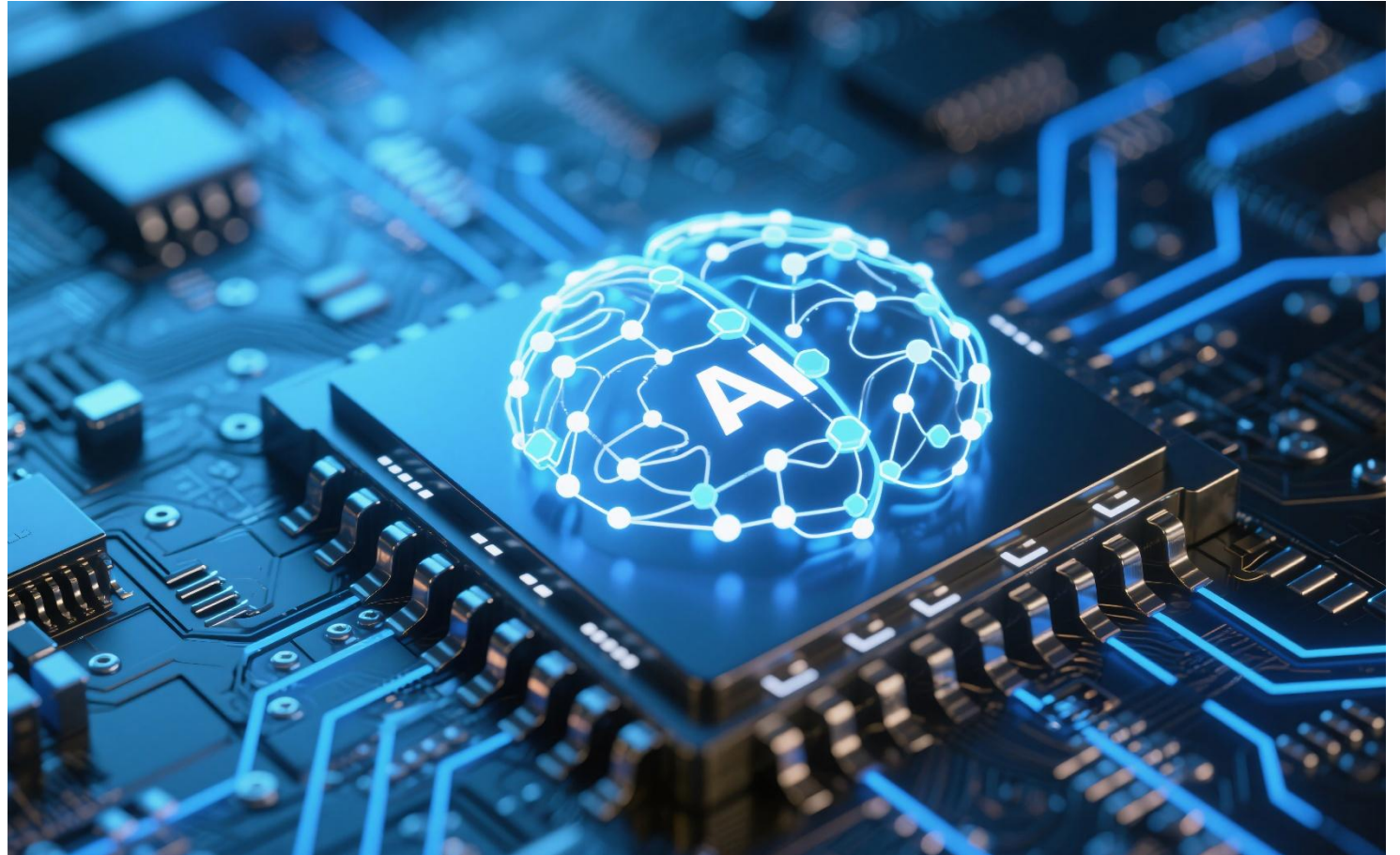
Test agents directly from your IDE. Access agent API via VS Code extension. Iterate on system messages and prompts without leaving editor.

- VS Code extension for Azure
- Test agent endpoints directly
- Local development loop

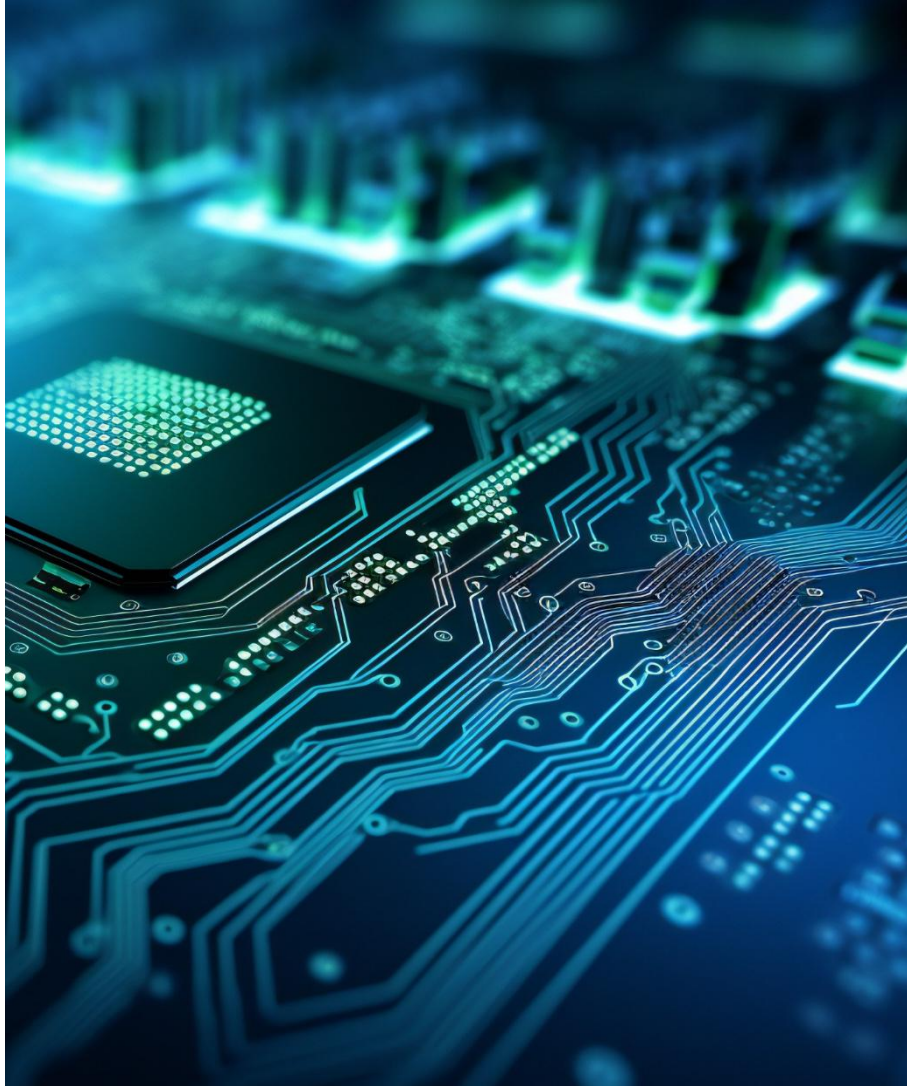
Lab 1.2: Azure Portal Setup

Log into Azure Portal with MFA. Explore subscriptions and resource groups. Understand resource organization. Navigate to AI Foundry.

- Login with credentials and MFA
- Locate and explore resource groups
- Find AI Foundry in portal



Lab 1.2: Upload to Blob Storage



Create a container in blob storage. Upload a sample CSV (customer reviews or support tickets). Verify upload and understand blob URI structure.

- Create container and set access level
- Upload CSV file to blob
- Note blob URL for RAG lab

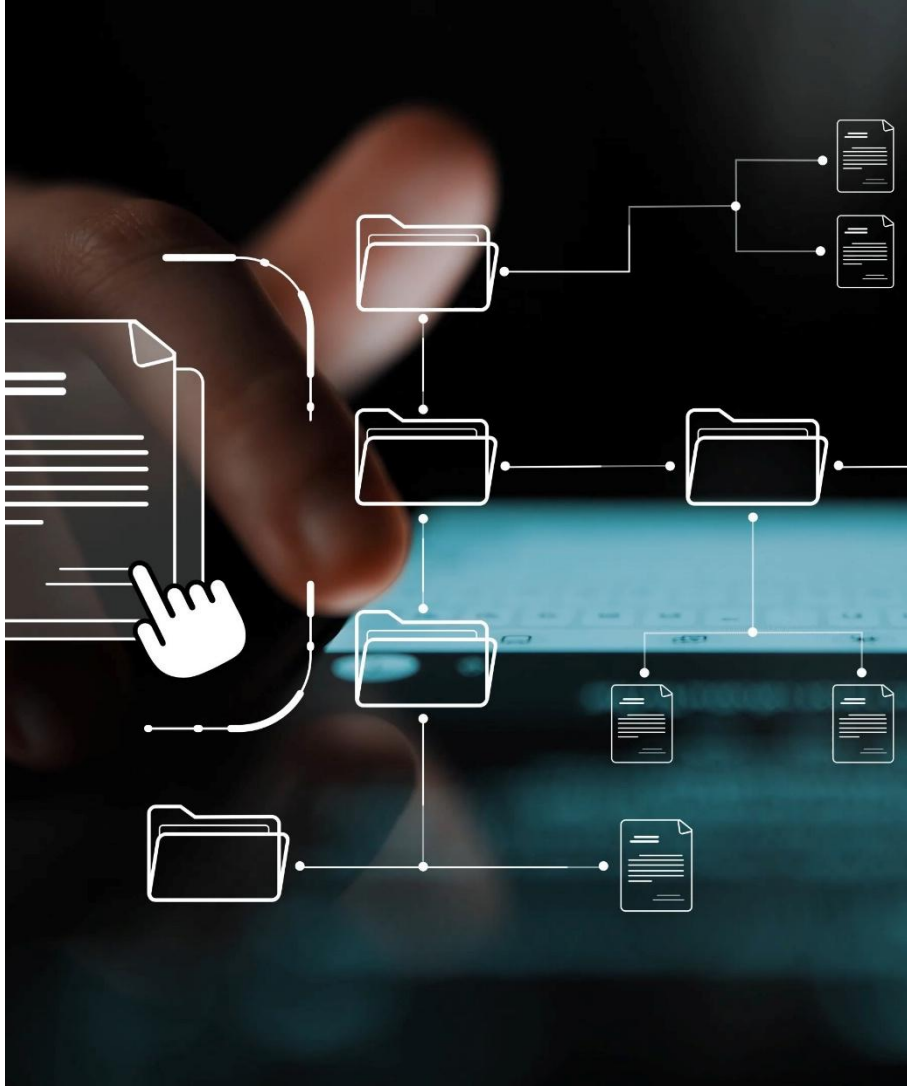
Lab 1.2: Build Your First Agent

Create an agent in Foundry. Write a system message defining role and behavior. Save the agent. Deploy as an endpoint.

- Define agent system message
- Configure basic settings
- Save and deploy agent



Lab 1.2: Test Your Agent



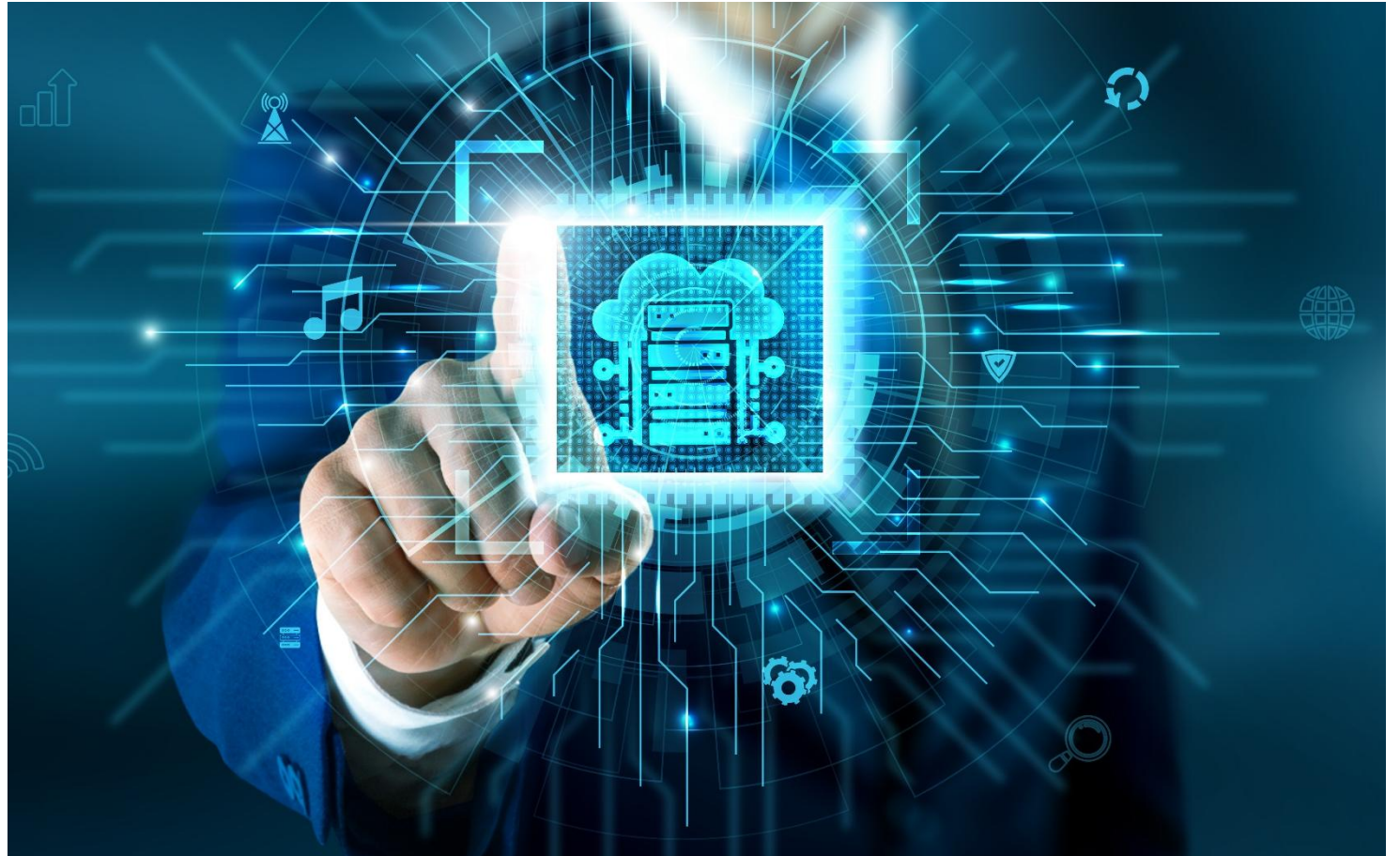
Chat with your deployed agent. Test different prompts. Observe response quality and latency. Tune temperature and system message based on results.

- Send test prompts to agent
- Observe response quality
- Adjust settings iteratively

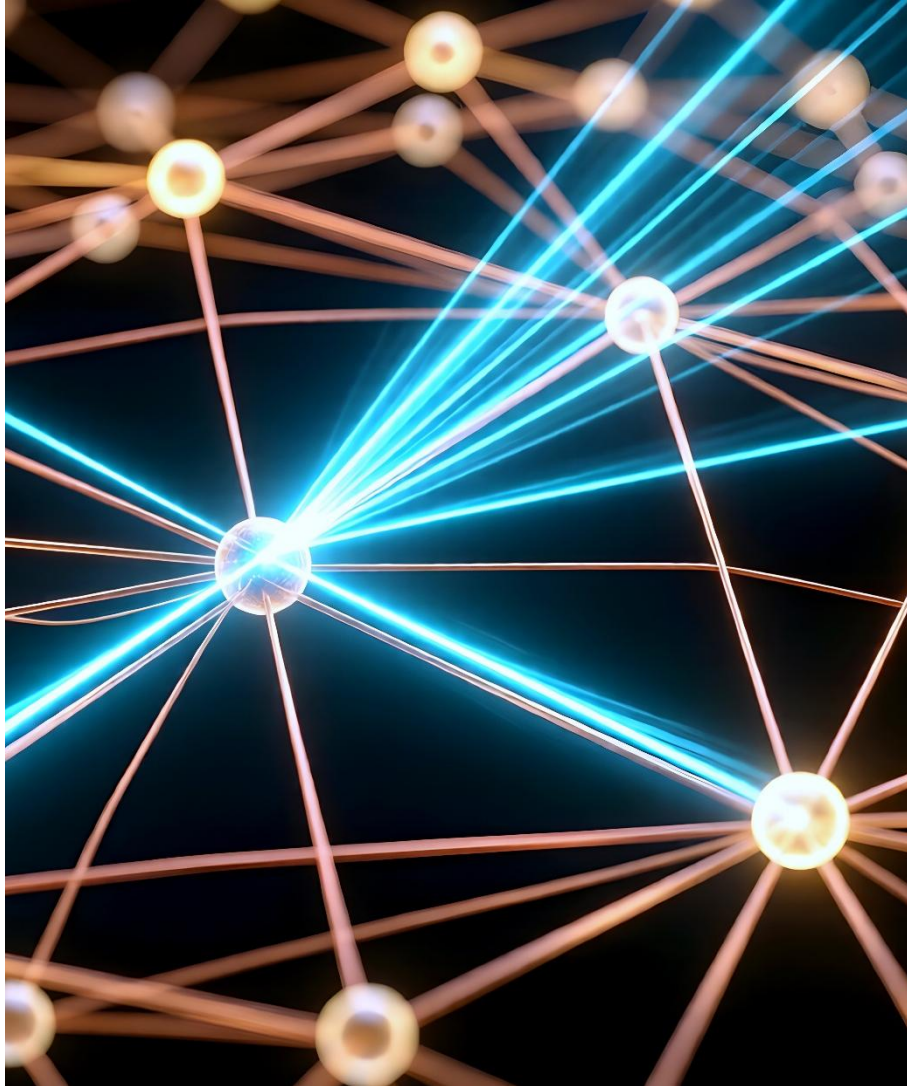
Lab 1.2 Summary

Key takeaways: Azure provides enterprise governance. Resource hierarchy enables cost control. Blob storage stages data for agents.

- Azure governance vs consumer tools
- Resource organization and RBAC
- Agent creation and deployment



Agent Configuration Overview



Control agent behavior through configuration: temperature, Top-P, memory, guardrails, knowledge sources. No code changes needed; all via Foundry UI.

- Temperature and sampling control
- Memory and conversation management
- Guardrails and knowledge sources

File Upload and Knowledge

Upload documents, PDFs, CSVs to agent.
Agent can reference files in responses.
Useful for domain-specific knowledge
without RAG indexing.

- Upload docs directly to agent
- Agent references files in context
- Good for small doc sets



Temperature Deep Dive



Scale 0.0-2.0 controls probability distribution over next tokens. 0.0: always pick highest probability (deterministic). 2.0: flatten distribution, equal chance for all.

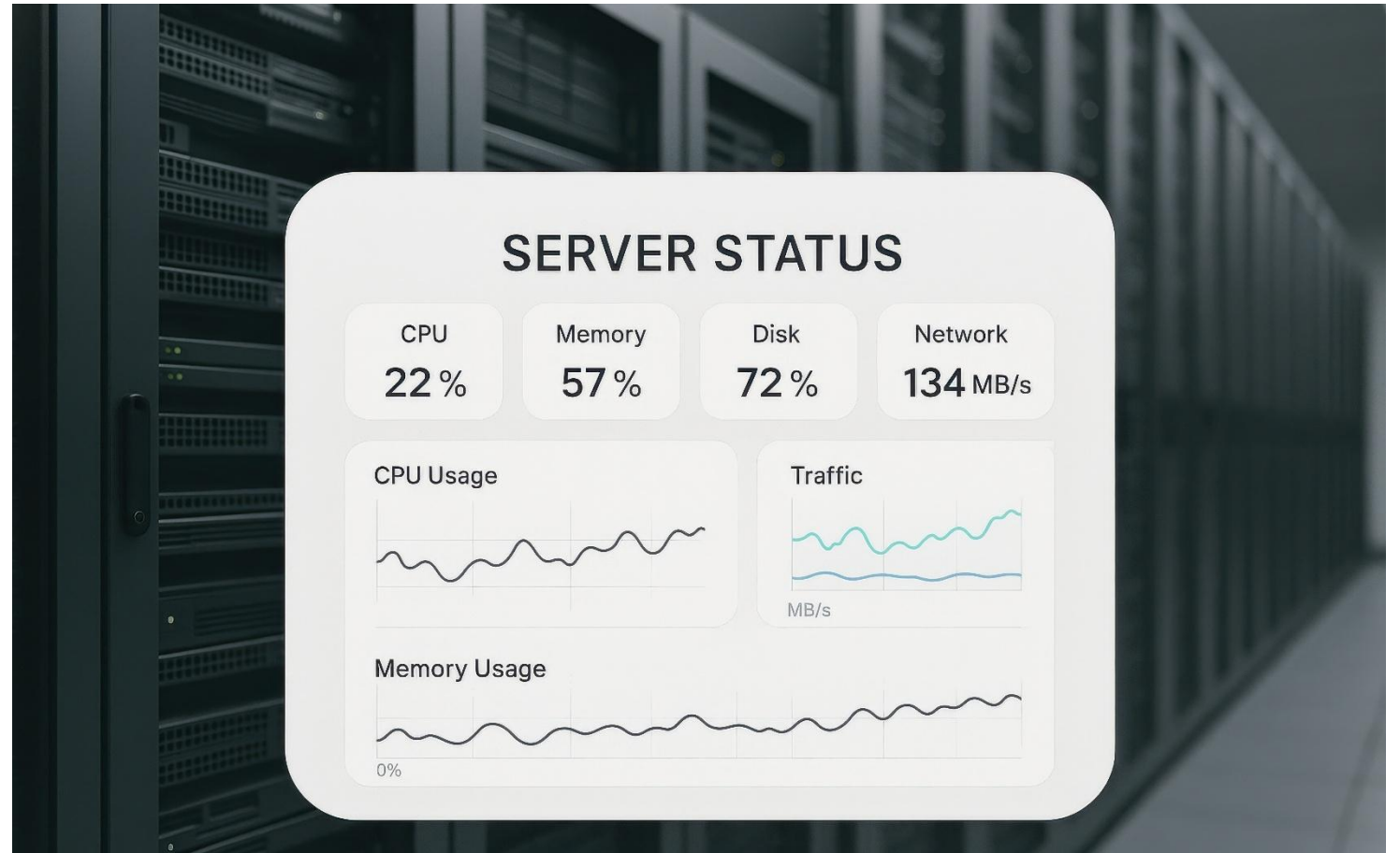
- 0.0: Greedy sampling, peak probability
- 1.0: Balanced entropy and predictability
- 2.0: Uniform over broader token set

Temperature in Practice

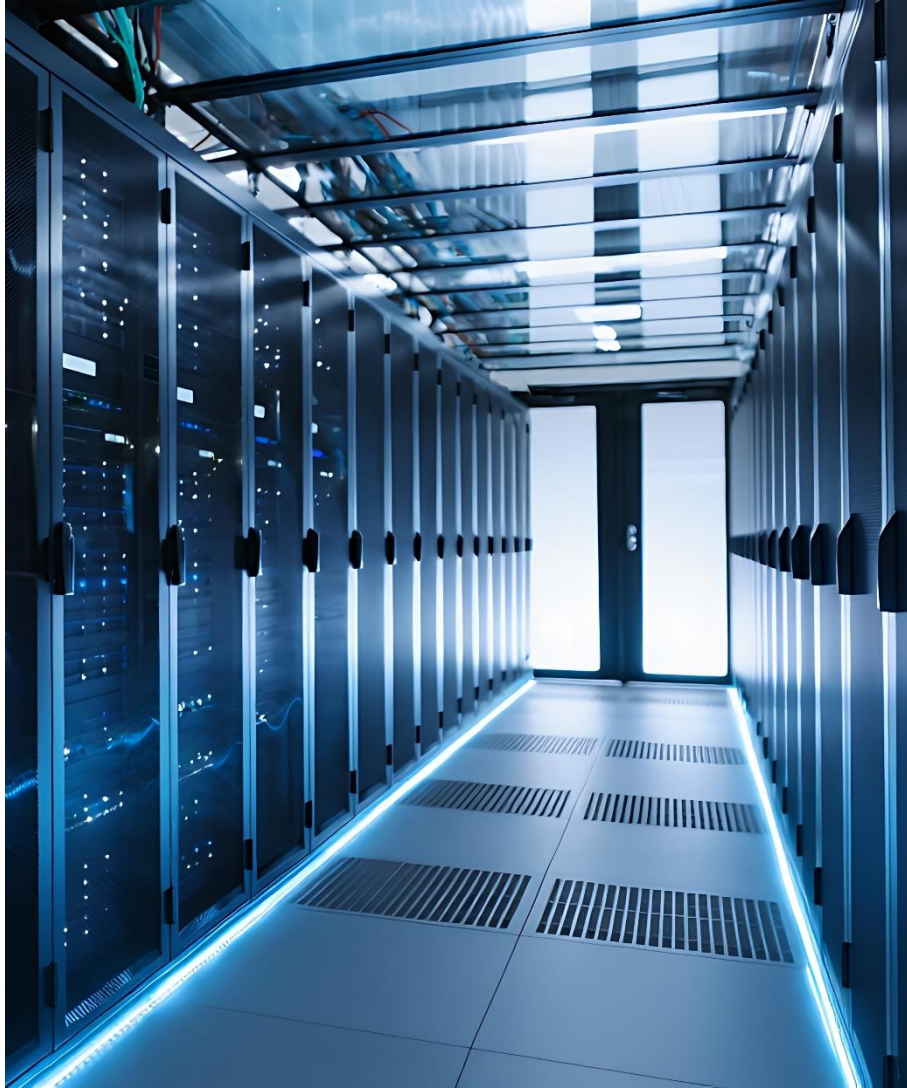
Low (0.1-0.3): Factual and consistent.

Medium (0.7-0.9): Balanced conversations. High (1.5-2.0): Creative and varied brainstorming.

- 0.1: Highly factual and consistent
- 0.7: Default, conversational tone
- 1.5: Creative, varied responses



Top-P Nucleus Sampling



Alternative to temperature. Top-P limits tokens to those whose cumulative probability reaches P . 0.9 means 90% of probability mass. Often combined with temperature.

- Top-P=0.9: 90% of probability mass
- Tighter control than temperature alone
- Commonly used in production

Temperature vs Top-P

Temperature: global randomness control. Top-P: token pool size. Together they shape response variety and predictability.

- Temp controls distribution shape
- Top-P controls token selection pool
- Combine for fine-tuned behavior



Agent Memory



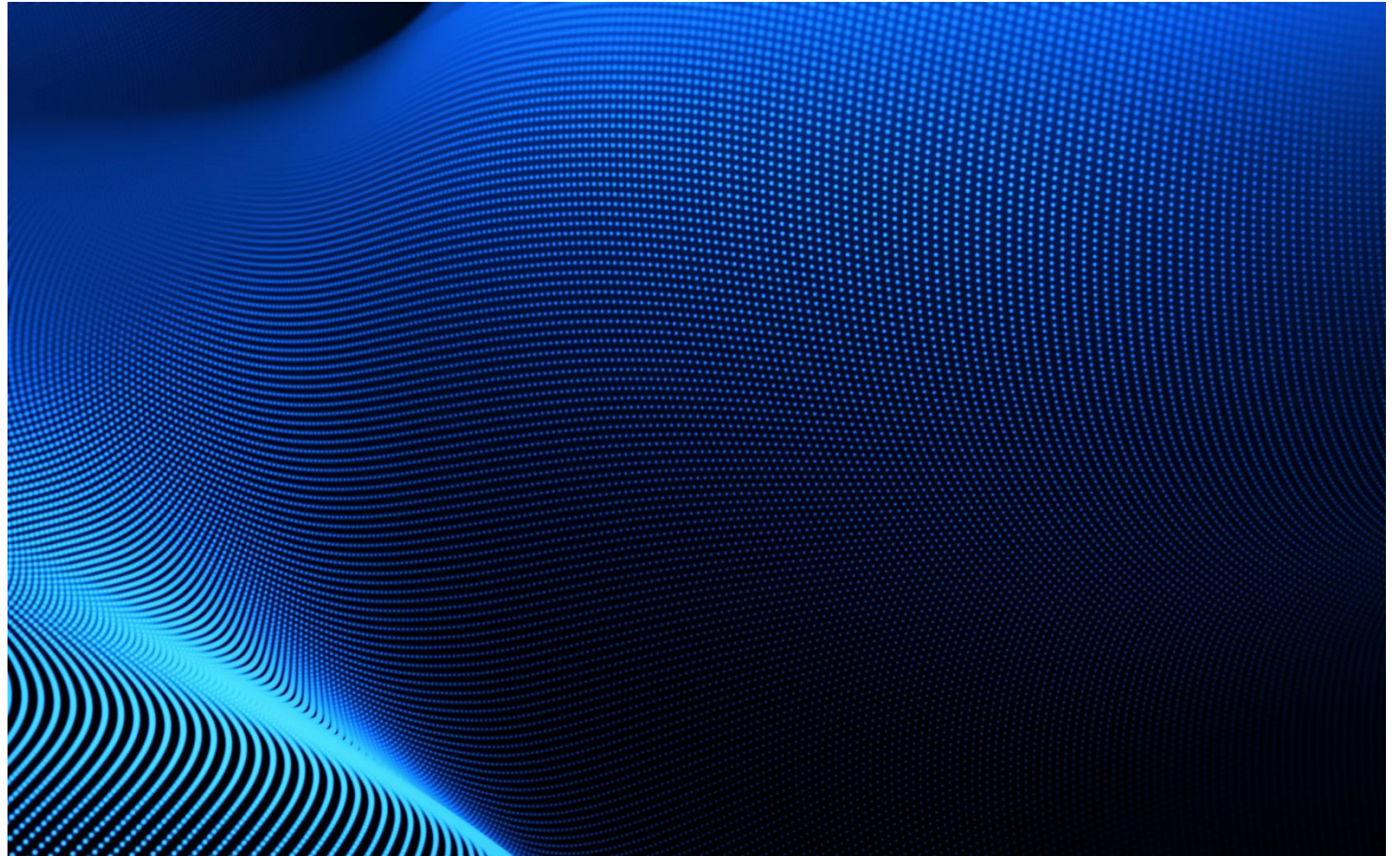
Conversation history kept in context. Session isolation: each conversation separate. Memory limits determined by context window. Long conversations exceed token budget.

- Keep recent messages in context
- Sessions expire after inactivity
- Memory trades off with cost

Guardrails and Knowledge Sources

Guardrails: system-level safety rules.
Knowledge sources: RAG indices or file uploads. Both shape agent behavior without code changes.

- Guardrails: content filters, jailbreak
- Knowledge: RAG indices, uploaded docs
- Foundry manages all connections



Agent Tracing



Full execution chain visible: user prompt → system message → tool calls → responses → latency → token counts. Traces enable debugging, security analysis, and optimization.

- Trace every user interaction
- See tool calls and LLM responses
- Debug execution issues step-by-step

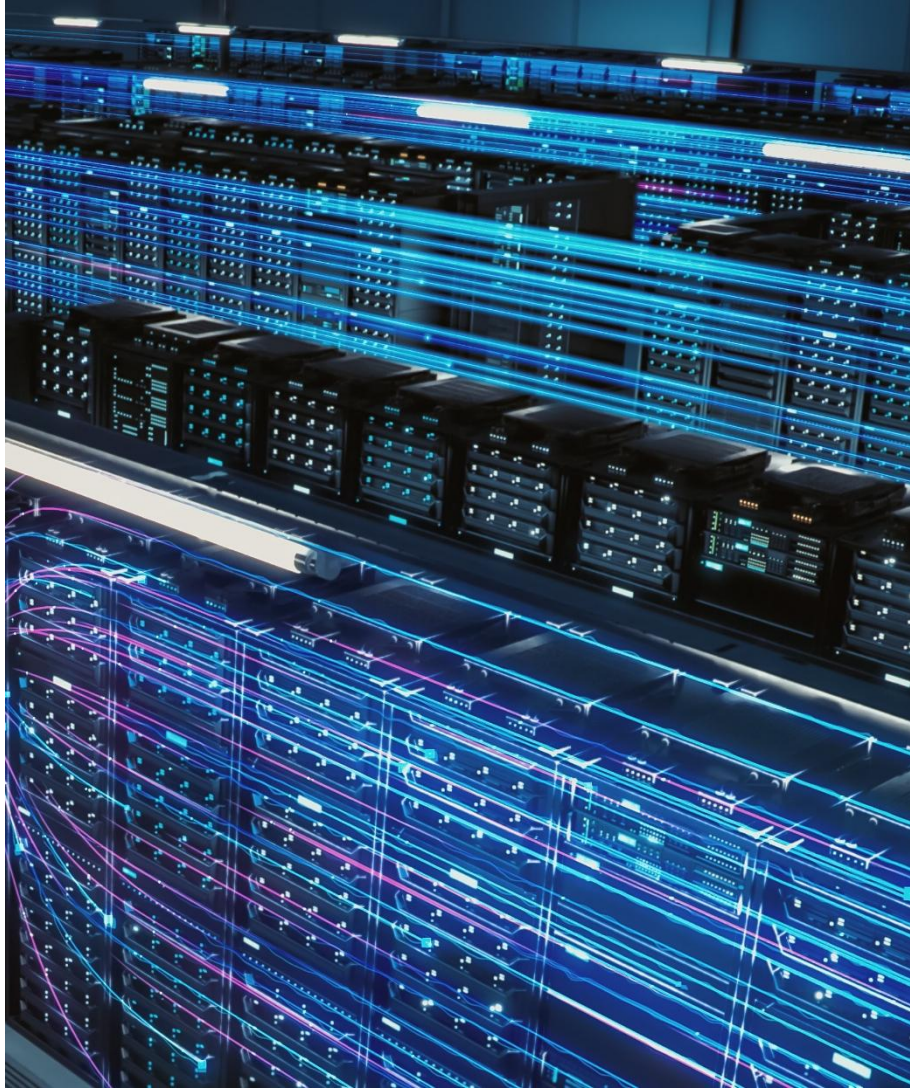
Tracing for Security

Audit trail of all interactions. Detect prompt injection attempts, data access patterns, policy violations. Essential for compliance audits.

- Complete audit history
- Detect injection and attacks
- Support compliance requirements



Operational Metrics Dashboard



Monitor requests per minute, latency percentiles, error rates, token consumption. Alerts for anomalies. Capacity planning based on metrics.

- Real-time request metrics
- Latency and error tracking
- Token consumption monitoring

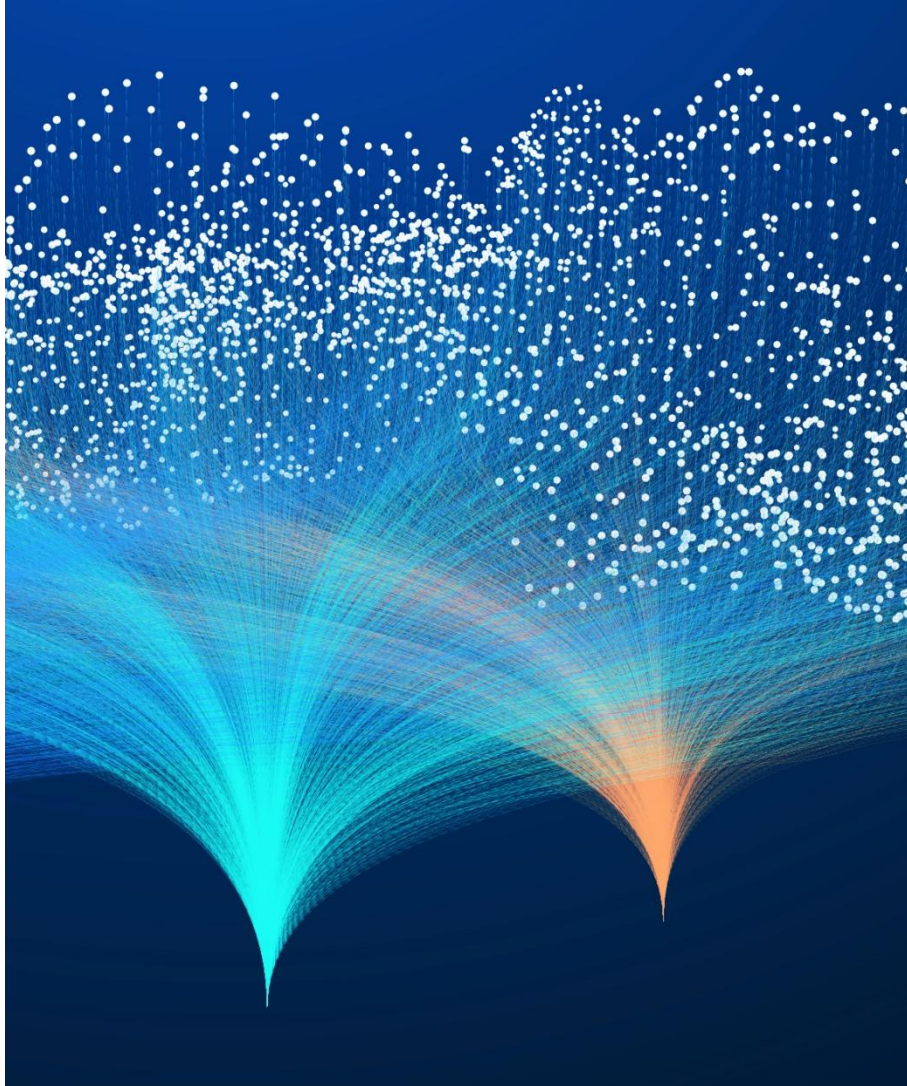
Cost and Risk Monitoring

Token budgets and cost limits prevent runaway spending. Content filter rates track safety. Risk dashboards flag violations.

- Token cost per request tracked
- Safety filter hit rates monitored
- Budget alerts and limits



Evaluations: AI Unit Tests



Structured tests for agent quality, safety, and accuracy. Groundedness: is response based on knowledge? Relevance: does it answer the question? Coherence: is it readable?

- Unit test analogy for AI systems
- Measure groundedness, relevance, safety
- Run on every deployment

Advanced Configuration

Production patterns: multi-turn design, error handling, tool limits, fallback strategies to prevent common failures.

- Design conversation flows carefully
- Handle errors gracefully
- Limit tool calls to prevent loops



Lab 1.3: Configure Your Agent



Adjust agent settings: temperature, Top-P, memory window, system message. Upload sample documents. Redeploy and re-test. Observe behavior changes.

- Adjust temperature and Top-P
- Upload knowledge files
- Redeploy and re-test agent

Lab 1.3: Temperature Experiment

Deploy same agent with temperature 0.1 and 1.0. Compare responses to identical prompts. Document consistency and creativity differences.

- Test identical prompts at 0.1 and 1.0
- Compare output consistency
- Document quality trade-offs



Lab 1.3: Explore Traces



Inspect trace for multi-turn conversation. See system message, tool calls, latency, tokens. Understand execution flow. Debug unexpected responses.

- View full conversation trace
- Inspect system message impact
- See tool calls and responses

Lab 1.3 Summary

Key takeaways: Configuration controls behavior without code. Temperature tunes randomness. Tracing enables debugging.

- Configuration is powerful and accessible
- Temperature matters for output quality
- Traces enable debugging



The Knowledge Problem



LLMs have a knowledge cutoff. They don't know your proprietary data, internal docs, or real-time information. They can't access external systems without special integration.

- LLMs: April 2024 knowledge cutoff
- No access to internal databases
- No real-time information

Why RAG?

Retrieve relevant documents, then generate grounded responses. Cheaper than fine-tuning, works with your latest data.

- Retrieves relevant documents first
- Grounds responses in knowledge base
- Updates without retraining



Semantic vs Keyword Search



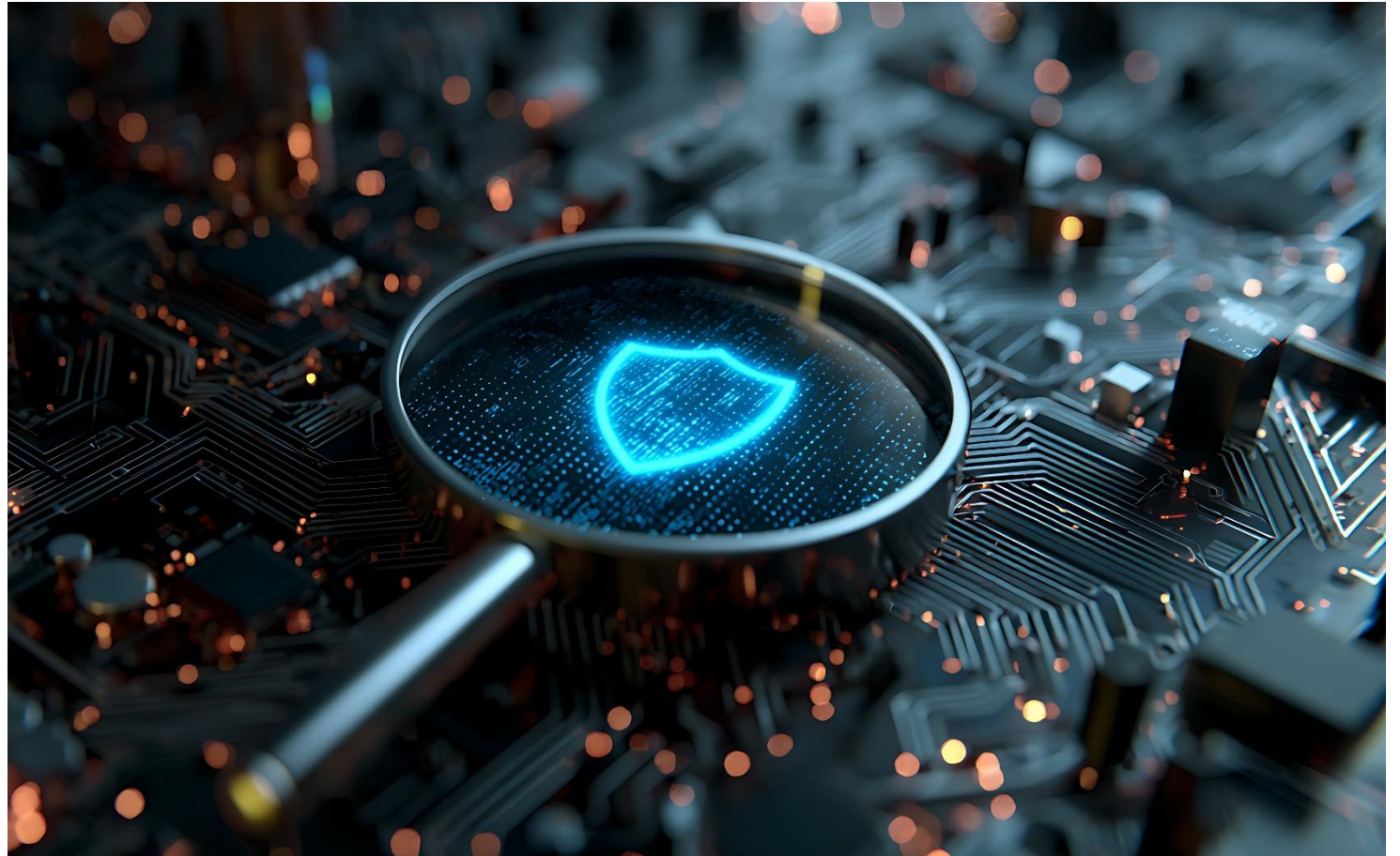
Keyword: exact word matching ('terrible support' won't match 'bad service'). Semantic: meaning-based ('terrible support' matches 'bad service'). Semantic is better for natural language.

- Keyword: exact lexical match
- Semantic: meaning-based matching
- RAG uses semantic search

What Is Vectorization?

Text becomes dense numerical vectors (e.g., 1536 dimensions). Similar texts produce similar vectors for fast search.

- Text → 1536-dimensional array
- Similar texts = close vectors
- Enables similarity computation



How Vectors Enable Search



Query converted to vector. Find nearest vectors (closest documents) using cosine similarity or Euclidean distance. Returns documents most similar in meaning to query.

- Query becomes vector
- Cosine similarity finds nearest neighbors
- Returns ranked list of documents

The Vectorization Process

Chunk documents into overlapping pieces, vectorize each chunk, index for fast retrieval, then query by similarity.

- Chunk documents (500-1000 tokens)
- Vectorize each chunk
- Index for fast search



Search Index Architecture



Specialized databases for vector search (not general SQL). Indexes vectors for fast nearest-neighbor search. Examples: Azure AI Search, Pinecone, Weaviate, Milvus.

- Optimized for vector similarity
- Fast retrieval: <100ms typical
- Scalable to millions of documents

Azure AI Search Setup

Create search service. Import documents from blob storage. Configure vectorizer (text to embeddings). Build search index. Query via REST API.

- Create search service in Azure
- Import data from blob
- Configure and build index



Index Field Configuration



Retrievable: return in results. Filterable: use in WHERE clause. Searchable: full-text search. Facetable: aggregate for filtering. Sortable: order results by field.

- Retrievable: include in results
- Searchable: include in text search
- Filterable: support WHERE clauses

Advanced RAG Settings

Semantic ranker reranks results using ML. Scheduled indexing refreshes data. Chunking strategy affects quality.

- Semantic ranker improves relevance
- Schedule indexing for freshness
- Tune chunk overlap



The RAG Agent Flow



User question → Agent retrieves documents → Agent generates response citing sources. Agent uses retrieved docs as context for generation. Enables grounded, source-aware responses.

- User query enters agent
- Agent retrieves matching documents
- Agent generates response with citations

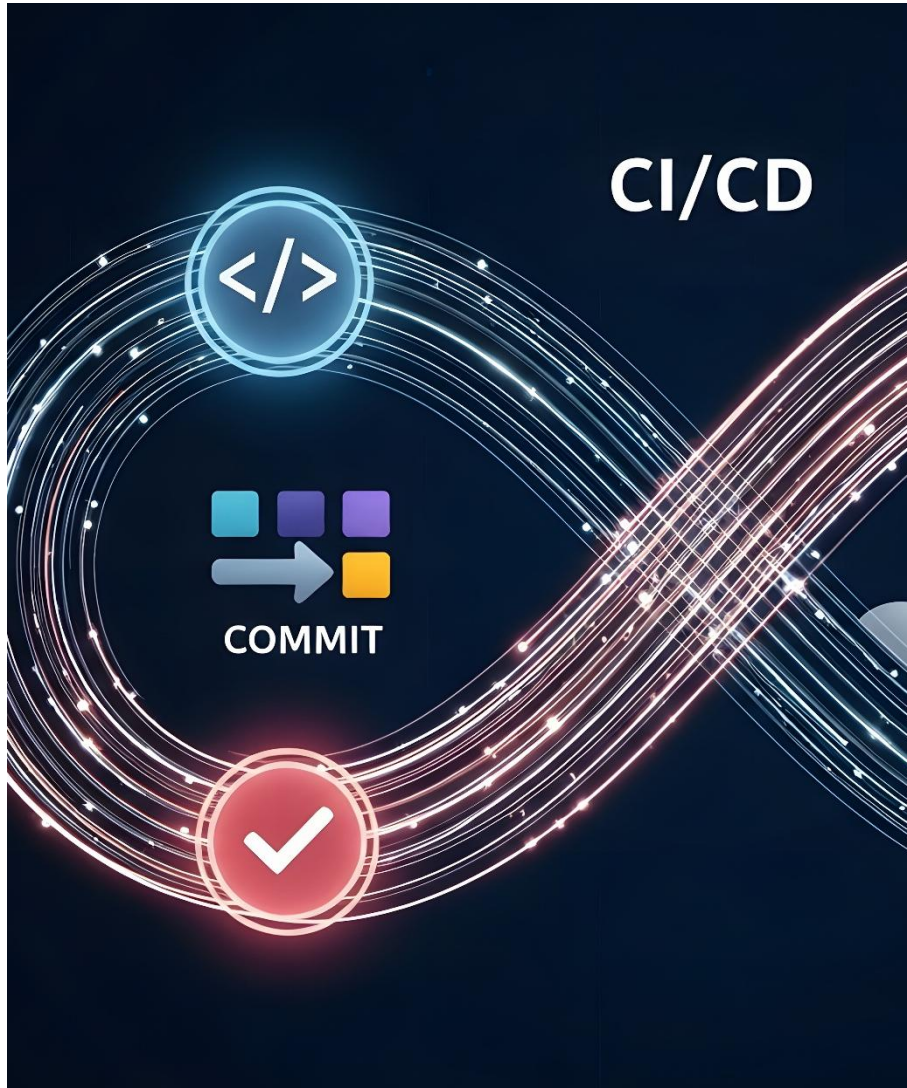
Agent Instructions for RAG

System message instructs agent to search its knowledge base first. Define citation format and accuracy expectations.

- 'Always search before answering'
- Define citation format explicitly
- Specify accuracy requirements



RAG Security Considerations



Indirect prompt injection: malicious text in documents could manipulate agent. Data poisoning: attackers inject false data into knowledge base. Filter documents and validate sources.

- Injection via document content
- Poisoning with false data
- Validate source credibility

Lab 1.4: Verify Blob Data

Confirm CSV in blob storage from Lab 1.2. Check file size, last modified date. Understand blob URI structure for next steps.

- Verify blob storage has CSV
- Check file properties
- Note blob URI



Lab 1.4: Create Search Index



Import blob CSV into Azure AI Search. Configure data source, indexer, and skill. Configure field properties (retrievable, searchable, filterable).

- Create data source from blob
- Create indexer and skill
- Configure fields appropriately

Lab 1.4: Vectorize and Build

Run vectorization skill on documents.
Build search index with vectors. Monitor indexing progress. Wait for completion before testing.

- Run vectorizer skill
- Monitor indexing progress
- Verify index creation



Lab 1.4: Connect Agent to RAG

SYSTEM LOG V2.5.1 - 04/23/1986

```
[08:15:21] USER 'ADMIN' LOGGED IN SUCCESSFULLY FROM TERMINAL #3.
[08:15:25] INITIALIZING MODULE 'CORE_SERVICES.MOD' -- STATUS: OK
[08:15:30] INITIALIZING MODULE 'NETWORK_CLIENT.MOD' -- STATUS: PING_F
[08:15:35] INITIALIZING MODULE 'NETWORK_CLIENT.MOD' -- STATUS: OK. CO
[08:15:40] LOADING CONFIGURATION FILE 'C:\SYSTEM\CONFIG.INI' FROM DIS
[08:15:45] LOADING MODULE 'DATABASE_MANAGER.MOD' -- STATUS: OK. CONNE
[08:15:50] QUERY: SELECT * FROM 'INVENTORY' WHERE 'ID' = '245'. 1 ROW
[08:16:00] DISPLAYING RESULTS TO TERMINAL #3.
[08:16:15] USER 'ADMIN' ISSUED COMMAND: 'VIEW LOGS'. ACCESS GRANTED.
[08:16:30] USER 'ADMIN' LOGGED OUT. SESSION TERMINATED.
[08:16:35] SYSTEM IDLE. WAITING FOR NEW USER LOGIN.
```

+-----+
| INVENTORY REPORT APRIL 1986
+-----+

ID	ITEM DESCRIPTION	QUANTITY	LOCATION	STATUS
01	PENTIUM PROCESSOR 8086	000185	A1.04	IN STOCK
02	MFM HARD DISK DRIVES	000003	A1.01	LOW STOCK
03	5.25" FLOPPY DISKETTES	001250	B3.07	AVAILABLE
04	CRT MONITORS (GREEN)	000012	C2.11	IN STOCK
05	KEYBOARD SETS W/ CABLE	000020	C2.15	IN STOCK
06	POWER SUPPLY UNITS	000007	A1.04	LOW STOCK
07	RS-232 CABLES	000500	B3.08	IN STOCK

PAGE 1 c

Volume in drive C has no label
Volume Serial Number is A8E0-607B

Directory of C:\

12/11/1985	08:30 AM	<DIR>	DATA
12/11/1985	08:31 AM	<DIR>	DOCS
12/11/1985	08:32 AM	<DIR>	GAMES
12/11/1985	08:35 AM	<DIR>	SYSTEM

Add search index as knowledge source to agent.
Update system message to instruct search. Test agent with questions about indexed data.

- Add index to agent knowledge
- Update system prompt
- Test RAG retrieval

Lab 1.4 Summary

Key takeaways: RAG grounds responses in real data. Semantic search via vectors is powerful. Azure AI Search integrates easily.

- RAG: retrieve then generate
- Vectors enable semantic search
- Azure handles infrastructure



What Is MCP?



Model Context Protocol: universal adapter between AI agents and external systems. Agents call MCP servers to access live data, execute commands, or integrate with tools.

- MCP: standardized integration protocol
- Agent ↔ MCP server ↔ real systems
- Live data and tool access

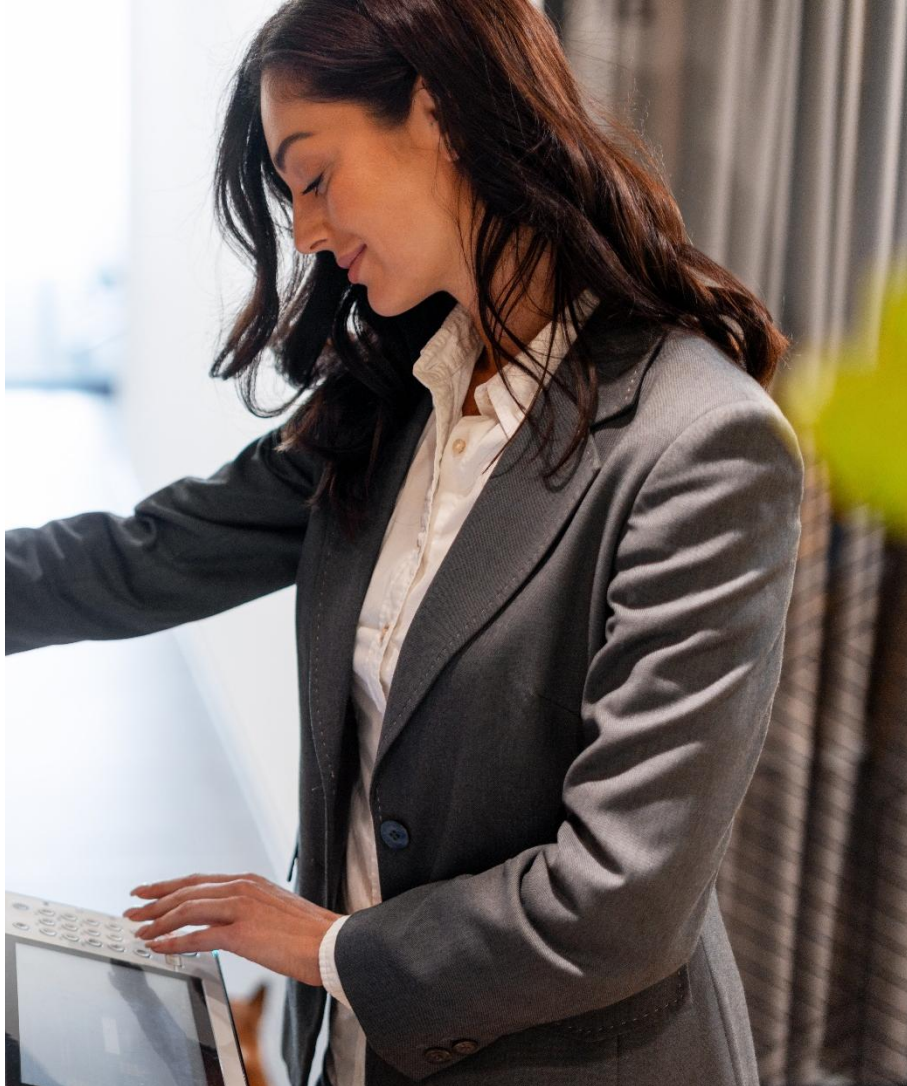
MCP vs RAG

RAG: static knowledge (documents, PDFs). MCP: dynamic systems (databases, APIs, services). RAG scales documents; MCP enables real-time integration.

- RAG: knowledge base documents
- MCP: live databases and APIs
- Use both together for complete



MCP Architecture



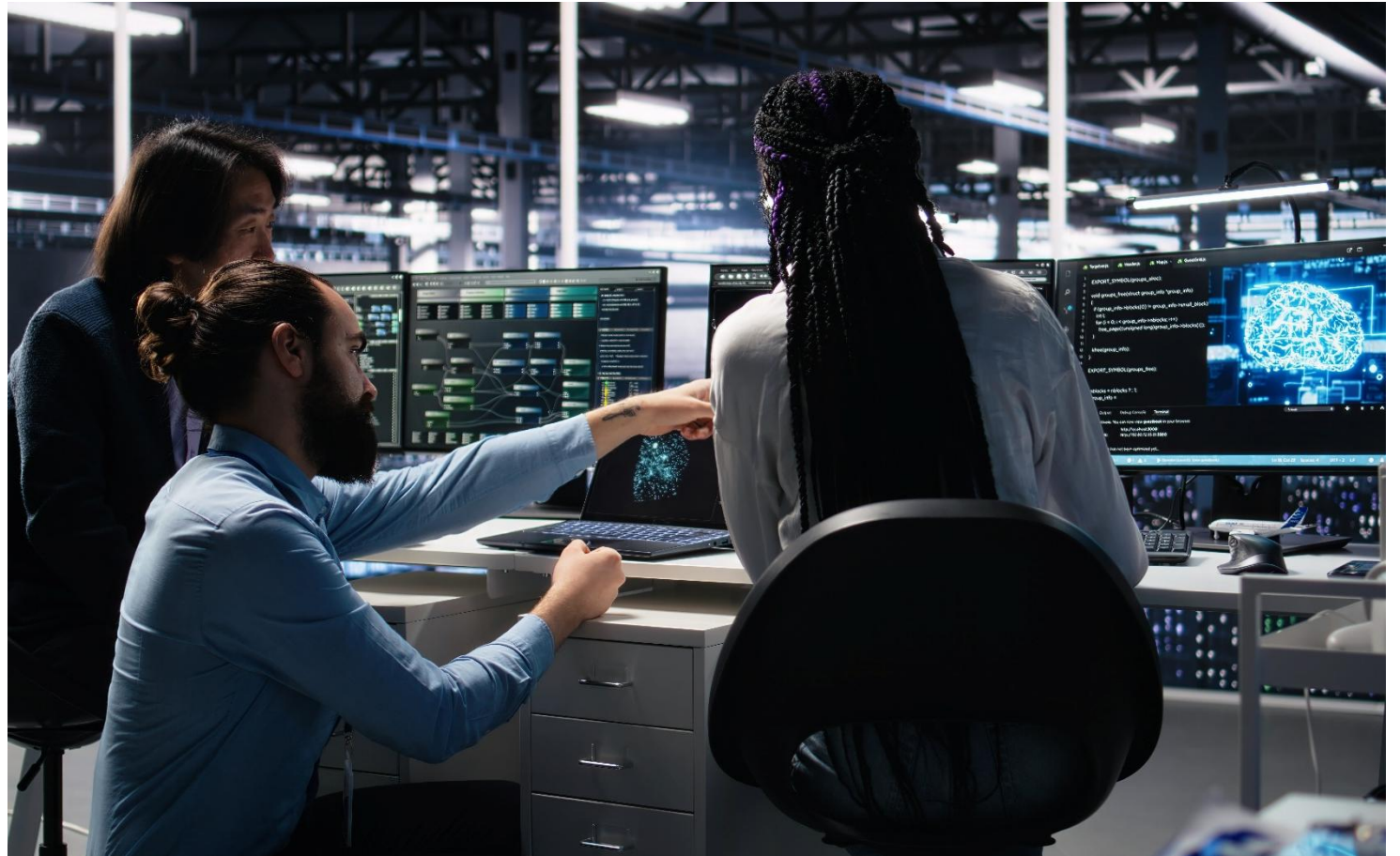
Agent sends requests to MCP server. Server executes against backend system (database, API, service). Server returns results to agent. Agent uses results in response.

- Agent initiates request
- Server accesses backend system
- Server returns structured result

MCP Security Boundary

MCP server is a gatekeeper. It controls what agent can access, which commands run, and what data is returned.

- Server enforces access control
- Server logs all requests
- Server validates all inputs



MCP JSON Configuration



Define MCP server in JSON: command (how to start), args, env vars, working directory. Agent loads config and starts server as subprocess.

- Define server command and args
- Set environment variables
- Configure working directory

Credential Management

Never hardcode credentials. Use environment variables or credential stores. MCP server reads creds from secure sources.

- Use env vars for secrets
- Never hardcode in config
- Use credential stores if available



SQL Injection Risk via MCP



Agent generates SQL queries. Malicious agent (or compromised system) injects SQL code. Server must parameterize queries, never string concatenation.

- Agent generates query intent
- Server must use parameterized queries
- Never concatenate user input

Live Data Queries

Agent asks in natural language. MCP translates to SQL. Database returns results. Agent sees live, real-time data.

- Agent: natural language question
- Server: SQL translation and execution
- Database: returns live results

